

Zero to Hero Study Handbook: granite-lora-qlora-text-classification

This handbook is based on static analysis of the repository files only:

- `granite41_lora_qlora_text_finetuning.ipynb`
- `scripts/run_granite41_gpu_stages.sh`
- `scripts/gpu_guard.sh`
- `requirements.txt`
- `README.md`
- `artifacts/*/adapter_config.json`
- `.gitignore`

No project code was executed for this analysis.

Module 1: Foundations & Architecture

1.1 What this project does

This repository implements an end-to-end text classification pipeline using IBM Granite 4.1 3B with:

- Zero-shot baseline classification
- LoRA fine-tuning (`peft.LoraConfig`)
- QLoRA fine-tuning (4-bit NF4 quantization via `BitsAndBytesConfig`)
- Generation-based evaluation (`evaluate_generation_classifier`)
- Adapter save/reload for inference (`PeftModel.from_pretrained`)

Primary use case in the code is AG News 4-class classification with labels:

- World
- Sports
- Business
- Sci/Tech

1.2 Core paradigms and patterns used in this repo

1. Notebook-centric ML pipeline

- The main runtime is a Jupyter notebook (`granite41_lora_qlora_text_finetuning.ipynb`), not a Python package with modules.

2. Stage-gated execution

- Execution is controlled by `RUN_STAGE` with valid values: `{"all", "base", "lora", "qlora", "final"}`.
- Gate function: `should_run_stage(stage_name: str) -> bool`.

3. Parameter-efficient fine-tuning (PEFT)

- LoRA adapters are trained while the base model stays frozen.

- QLoRA reuses LoRA adapters on a quantized 4-bit base model.
4. **Generative classification pattern**
 - Instead of logits over classes, the model generates label text.
 - Decoding/parsing path: `predict_label(...)` -> `normalize_predicted_label(...)`.
 5. **Memory-first training design**
 - Uses `gradient_checkpointing_enable`, `model.config.use_cache = False`, bf16/fp16 selection, and aggressive cleanup (`cleanup_torch_objects`).
 6. **Operational shell orchestration**
 - `scripts/run_granite41_gpu_stages.sh` runs notebook stages sequentially using `jupyter-nbconvert`.
 - OOM retry strategy reduces sequence length via predefined ladders.
 - `scripts/gpu_guard.sh` clears stale Python GPU processes and waits for free VRAM.

1.3 Architecture: components and interactions

Key components

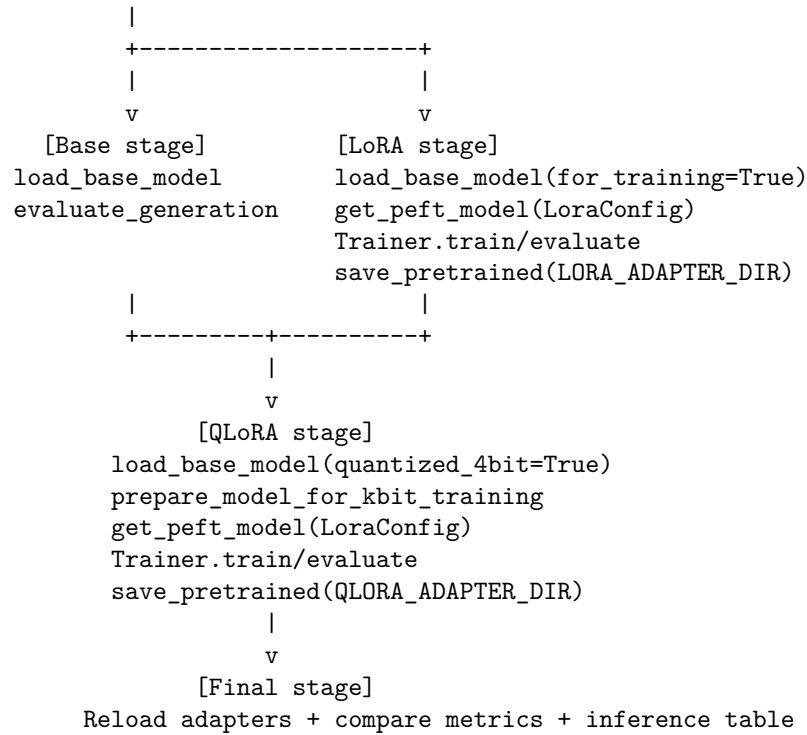
- **Main pipeline:** `granite41_lora_qlora_text_finetuning.ipynb`
- **Stage runner:** `scripts/run_granite41_gpu_stages.sh`
- **GPU preflight guard:** `scripts/gpu_guard.sh`
- **Model artifacts:** `artifacts/lora_adapter`, `artifacts/demo_lora_adapter`, `artifacts/demo_qlora_adapter`
- **Dependency manifest:** `requirements.txt`

Main flow (ASCII)

```

[User: Jupyter or nbconvert]
  |
  v
[Notebook config + env parsing]
- RUN_STAGE
- FAIL_ON_STAGE_ERROR
- MAX_SEQ_LENGTH_OVERRIDE
  |
  v
[Dataset load + split]
load_dataset -> train_ds/val_ds/gen_eval_ds
  |
  v
[Prompt + tokenization]
build_prompt
encode_supervised_example
causal_lm_collator

```



Automation flow (ASCII)

```

run_granite41_gpu_stages.sh
-> run base once (seq=384)
-> run lora with retry ladder (384,320,256,192,160,128,96,64)
-> run qlora with retry ladder (384,320,256,192,160,128)
-> run final once (seq=384)

```

For each attempt:

```

gpu_guard.sh -> ensure VRAM target -> nbconvert with RUN_STAGE + MAX_SEQ_LENGTH_OVERRIDE
if fail and log has OOM text -> retry with smaller seq

```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
granite41_lora_qlora_textfinetuning	training/eval notebook pipeline	args.py, should_run_stage, set_seed, get_compute_dtype, detect_runtime, memory_snapshot, cleanup_torch_objs, build_prompt, normalize_predictions, encode_supervised_queries, causal_lm_collator, load_base_model, predict_label, evaluate_generation_classifier, make_training_args, print_trainable_ratio	RUN_STAGE, FAIL_ON_STAGE_ERROR, MAX_SEQ_LENGTH_OVERRIDE, SEED, MODEL_ID, MAX_SEQ_LENGTH, NUM_EPOCHS, LEARNING_RATE, GRAD_ACCUM_STEPS, LORA_ADAPTER_DIR, LORA_ADAPTER_DIR, DEMO_*
scripts/run_granite41_textfinetuning	notebook-stage executor with OOM retry	run_stage_once, run_stage_with_retry, run_stage_single	shs_oom_failure, NB_PATH, JUPYTER_NB_CONVERT, GPU_GUARD_SCRIPT, VRAM_FREE_TARGET_MIB, GPU_GUARD_GRACE_SEC, GPU_GUARD_POLL_SEC, GPU_GUARD_TIMEOUT_SEC, RUN_ID, MAX_SEQ_LENGTH_OVERRIDE, RUN_STAGE
scripts/gpu_guard	CPU process cleanup and free-VRAM wait loop	list_python_gpu_processes, kill_python_gpu_processes, read_gpu_free_mib, wait_for_vram	positional args: TARGET_FREE_MIB, GRACE_SECONDS, POLL_SECONDS, TIMEOUT_SECONDS
requirements.txt	Python dependency pinning	N/A	torch==2.12.0, transformers==5.12.0, peft==0.19.1, datasets==5.0.0, bitsandbytes==0.49.2, accelerate==1.14.0
artifacts/lora_adapter	Adapter metadata for full run	N/A	json, peft_type=LORA, r=16, lora_alpha=32, lora_dropout=0.05, task_type=CAUSAL_LM, target_modules=[q_proj, k_proj, v_proj, o_proj]

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
artifacts/demo_lora_adapter/adapter_config.json	Saved adapter metadata for demo run	N/A	Same core LoRA config as above
artifacts/demo_quantized_adapter/adapter_config.json	Saved adapter metadata for demo run	N/A	Same LoRA adapter config on quantized base
artifacts/*/tokenizer_config.json	Tokenization settings saved with adapters	N/A	padding_side=left, pad_token=< pad >, eos_token=< end_of_text >
README.md	Human-facing tutorial narrative and run instructions	N/A	Installation commands, memory notes, reported metric tables
.gitignore	Exclusion policy for large/derived files	N/A	excludes .venv/, artifacts/*_run/, artifacts/runs/, *.safetensors, *.executed.ipynb

First files a new contributor should read

1. README.md (project intent and expected outcomes)
2. granite41_lora_qlora_text_finetuning.ipynb (true runtime logic)
3. scripts/run_granite41_gpu_stages.sh (non-interactive execution model)
4. scripts/gpu_guard.sh (GPU hygiene and VRAM gating)
5. artifacts/lora_adapter/adapter_config.json (actual saved adapter spec)

Module 3: Core Execution Flows

3.1 Flow A: Runtime and stage dispatch

Step-by-step

1. Global constants are defined (SEED, MODEL_ID, paths, training hyperparameters).
2. RUN_STAGE is read from environment and validated against VALID_RUN_STAGES.
3. ACTIVE_STAGES is derived from STAGE_EXECUTION_MAP.
4. Each major block checks should_run_stage("base"|"lora"|"qlora"|"final").
5. detect_runtime() determines CUDA and bitsandbytes support.

Important design detail

- Notebook sets: `PROJECT_ROOT = Path("/home/ahmad/AI/Notebooks/LoRA-QLoRA")`.
- This is not the current repo path, so users need to align this path before running.

3.2 Flow B: Data preparation and supervised encoding

Dataset loading path

```
DATASET_CANDIDATES = ["fancyzhx/ag_news", "ag_news"]
raw = load_dataset(ds_id)
```

Then:

- `train_full = raw["train"]`
- `test_full = raw["test"]`
- `train_val = train_full.train_test_split(test_size=10_000, seed=SEED, shuffle=True)`
- `train_ds = train_val["train"]`
- `val_ds = train_val["test"]`
- `gen_eval_ds = test_full.select(range(GEN_EVAL_MAX_SAMPLES))`
when configured

Input/output schema (exact shapes by code contract)

1. Raw dataset example (`train_ds` / `gen_eval_ds`)

```
{
  "text": str,
  "label": int # expected 0..3
}
```

2. Prompt construction

```
build_prompt(article_text: str) -> str
```

Template includes:

- instruction line
- article text
- trailing Label:

3. Training example encoding

```
encode_supervised_example(example: dict) -> dict
```

Returns:

```
{
  "input_ids": list[int],
  "attention_mask": list[int], # all 1 before padding
}
```

```

    "labels": list[int] # prompt positions = -100; target label tokens kept
}

```

4. Batch collation

causal_lm_collator(features: list[dict]) -> dict

Returns PyTorch tensors:

```

{
    "input_ids": Tensor[batch, seq_len],
    "attention_mask": Tensor[batch, seq_len],
    "labels": Tensor[batch, seq_len] # pad and prompt ignored via -100
}

```

3.3 Flow C: Base model generation evaluation

Step-by-step

1. load_base_model(quantized_4bit=False, for_training=False)
2. base_model.eval()
3. evaluate_generation_classifier(base_model, gen_eval_ds, ...)
4. Metrics stored in base_metrics; per-example rows in base_predictions.

Internal loop details For each example:

1. predict_label builds prompt and tokenizes.
2. Greedy decode via model.generate(..., do_sample=False, max_new_tokens=MAX_NEW_TOKENS_LABELS).
3. Generated suffix is decoded.
4. normalize_predicted_label maps raw string to canonical label or "UNKNOWN".

Returned tuple from evaluator:

```
(metrics: dict, predictions_df: pd.DataFrame)
```

metrics keys:

- run_name
- samples_evaluated
- accuracy_strict
- macro_f1_known_only
- label_parse_coverage

predictions_df columns:

- text
- true_label
- pred_label
- raw_generation

3.4 Flow D: LoRA training stage

Step-by-step

1. Clear previous refs: `base_model = None`, then `cleanup_torch_objects()`.
2. Load bf16/fp16 base with training flags: `load_base_model(quantized_4bit=False, for_training=True)`.
3. Build LoRA config:

```
LoraConfig(  
    r=16,  
    lora_alpha=32,  
    lora_dropout=0.05,  
    target_modules="all-linear",  
    bias="none",  
    task_type=TaskType.CAUSAL_LM,  
)
```

4. Wrap model: `lora_model = get_peft_model(lora_base_model, lora_config)`.
5. Build args: `lora_train_args = make_training_args(...)`.
6. Train with HF Trainer.
7. Evaluate token-level loss.
8. Save adapter + tokenizer: `save_pretrained(LORA_ADAPTER_DIR)`.
9. Run generation eval: `evaluate_generation_classifier(...)`.

TrainingArguments compatibility pattern `make_training_args` filters keys dynamically using:

- `inspect.signature(TrainingArguments.__init__)`
- fallback key selection between `eval_strategy` and `evaluation_strategy`

This avoids version breakage in Transformers 5.x.

3.5 Flow E: QLoRA training stage

Step-by-step

1. Clear LoRA objects to reclaim VRAM: `lora_trainer = None`;
`lora_model = None`; `lora_base_model = None`.
2. Validate runtime support: `runtime["qlora_supported"]`.
3. Load quantized base: `load_base_model(quantized_4bit=True, for_training=True)`.
4. Prepare k-bit training: `prepare_model_for_kbit_training(qlora_base_model)`.
5. Apply same `LoraConfig`.
6. Train with Trainer.
7. Save to `QLORA_ADAPTER_DIR`.
8. Run generation evaluation.

Quantization config used

```
BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=get_compute_dtype(),
)
```

3.6 Flow F: Final comparison and reload inference

Comparison path `comparison_rows` is assembled into `comparison_df` with columns:

- `run`
- `accuracy_strict`
- `macro_f1_known_only`
- `label_parse_coverage`
- `train_runtime_sec`
- `token_eval_loss`

If metrics are missing, final stage can recompute by reloading saved adapters.

Reload inference path

1. Reload clean base model.
2. Optionally attach LoRA adapter: `PeftModel.from_pretrained(lora_base_for_reload, str(LORA_ADAPTER_DIR))`.
3. Optionally attach QLoRA adapter with quantized base.
4. Run `predict_label` on three sample articles.
5. Build `inference_df`.

3.7 Flow G: Shell orchestration (non-notebook execution)

`scripts/run_granite41_gpu_stages.sh` executes notebook stages as separate `nbconvert` runs:

1. Stage `base` once at seq 384.
2. Stage `lora` with retry ladder.
3. Stage `qlora` with retry ladder.
4. Stage `final` once.

For each attempt:

- Calls `scripts/gpu_guard.sh` first.
- Sets env vars: `RUN_STAGE`, `FAIL_ON_STAGE_ERROR=true`, `MAX_SEQ_LENGTH_OVERRIDE=<seq_len>`.
- Executes: `jupyter-nbconvert --to notebook --execute ....`
- Writes logs under: `artifacts/runs/<RUN_ID>/`.

OOM detection logic:

- `is_oom_failure` searches logs with regex: `cuda out of memory|out of memory.`
- If matched, retries with smaller sequence length.
- Non-OOM failures abort retries.

Module 4: Setup & Run Guide

4.1 Dependencies and toolchain

From `requirements.txt` and scripts, this project expects:

- Python 3.12 (README uses `uv venv --python 3.12`)
- PyTorch + Transformers + PEFT + Datasets + bitsandbytes
- Jupyter + nbconvert + ipykernel
- NVIDIA stack for GPU path (`nvidia-smi` is required by `gpu_guard.sh`)

4.2 Clean-machine setup (repo-documented path)

```
git clone https://github.com/pypi-ahmad/granite-lora-qlora-text-classification.git
cd granite-lora-qlora-text-classification
```

```
uv venv --python 3.12
source .venv/bin/activate
uv pip install -r requirements.txt
```

Optional CUDA-specific torch install path from README:

```
uv pip install torch==2.12.0 --index-url https://download.pytorch.org/whl/cu121
uv pip install -r requirements.txt --no-deps
uv pip install -r requirements.txt
```

Jupyter kernel registration (README):

```
.venv/bin/python -m ipykernel install --user --name granite-lora-env --display-name "Granite"
```

4.3 Required/used environment variables

Notebook variables

Variable	Where used	Default	Purpose
<code>RUN_STAGE</code>	notebook	all	Stage gating (base, lora, qlora, final)
<code>FAIL_ON_STAGE_ERROR</code>	notebook	True	Fail-fast behavior for stage exceptions
<code>MAX_SEQ_LENGTH_OVERRIDE</code>	notebook	unset	Override <code>MAX_SEQ_LENGTH</code>

Variable	Where used	Default	Purpose
PYTORCH_CUDA_ALLOC_CONF	Code + README	expandable_segments (setdefault)	CUDA allocator behavior for fragmentation control

Stage-runner variables (`scripts/run_granite41_gpu_stages.sh`)

Variable	Default	Purpose
NB_PATH	<repo>/granite41_lora_qlora_text_finetuning.ipynb	Notebook to finetune
JUPYTER_NBCONVERT	<repo>/venv/bin/jupyter-nbconvert	Executable
GPU_GUARD_SCRIPT	<repo>/scripts/gpu_guard.sh	GPU preflight command
VRAM_FREE_TARGET_MIB	7600	Required free VRAM before each stage
GPU_GUARD_GRACE_SEC	8	Grace delay between kill and check
GPU_GUARD_POLL_SEC	2	VRAM polling interval
GPU_GUARD_TIMEOUT_SEC	0	0 = no timeout
RUN_ID	timestamp	Artifact/log run folder name

4.4 Typical command sequences

Interactive notebook path

1. Activate environment.
2. Open and run: `granite41_lora_qlora_text_finetuning.ipynb`.
3. Optionally set: `export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`.

Scripted stage execution path

```
chmod +x scripts/run_granite41_gpu_stages.sh scripts/gpu_guard.sh
scripts/run_granite41_gpu_stages.sh
```

Dry run:

```
scripts/run_granite41_gpu_stages.sh --dry-run
```

4.5 Configuration caveats to fix before real runs

1. In notebook code, `PROJECT_ROOT` is hardcoded to: `/home/ahmad/AI/Notebooks/LoRA-QLoRA`.
 - In this repository location, update it to the actual repo path or derive from CWD.
2. `scripts/run_granite41_gpu_stages.sh` expects executable `venv/bin/jupyter-nbconvert`.
 - Ensure virtual environment exists before using script mode.

3. `scripts/gpu_guard.sh` requires `nvidia-smi`.
 - On non-NVIDIA systems, scripted GPU guard flow will exit early.

4.6 Migrations / seeding / external services

- No database migrations are present.
- No DB seeding scripts are present.
- External services used by code:
 - Hugging Face model + dataset hubs (`AutoModelForCausalLM.from_pretrained, load_dataset`).
- Optional auth in README: `huggingface-cli login` (not mandatory for public assets).

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

1. **Read README.md first**
Goal: understand expected outcomes (Base vs LoRA vs QLoRA), memory targets, and constraints.
2. **Read notebook config + utilities section** (`granite41_lora_qlora_text_finetuning.ipynb`)
Focus on: `RUN_STAGE`, stage map, runtime detection, memory cleanup helpers.
3. **Read data and prompt pipeline functions**
Focus on: `build_prompt`, `normalize_predicted_label`, `encode_supervised_example`, `causal_lm_collator`.
4. **Read model loading + evaluation functions**
Focus on: `load_base_model`, `predict_label`, `evaluate_generation_classifier`.
5. **Read LoRA + QLoRA training blocks**
Focus on: `LoraConfig`, `prepare_model_for_kbit_training`, `make_training_args`, `Trainer` usage.
6. **Read final-stage comparison/reload blocks**
Focus on: adapter reload pattern and metric table assembly.
7. **Read shell scripts**
Focus on automation, OOM retries, and GPU process management.
8. **Inspect adapter configs in artifacts/**
Goal: confirm persisted training config matches notebook config.

5.2 Practice exercises

Exercise 1 Trace how a raw AG News sample becomes a supervised training item. Identify each function and the exact output keys.

Exercise 2 Explain why `labels` contains `-100` for prompt tokens in `encode_supervised_example`.

Exercise 3 List all `RUN_STAGE` values and explain what happens if an invalid value is provided.

Exercise 4 Compare LoRA and QLoRA model loading in `load_base_model`. Which arguments differ?

Exercise 5 Describe every field returned by `evaluate_generation_classifier` metrics dict and prediction DataFrame.

Exercise 6 Explain why `make_training_args` uses signature filtering and the `EVAL_STRATEGY_KEY` fallback.

Exercise 7 In `run_granite41_gpu_stages.sh`, describe exactly how OOM retry works for `lora` and `qlora`.

Exercise 8 Using `artifacts/*/adapter_config.json`, verify whether saved adapter hyperparameters match notebook LoRA config.

5.3 Solution outlines

Solution 1 `load_dataset` -> split to `train_ds/val_ds` -> `encode_supervised_example` -> dict with `input_ids`, `attention_mask`, `labels` -> `causal_lm_collator` pads into tensors for `Trainer`.

Solution 2 `-100` is the ignore index for cross-entropy. This masks prompt and pad positions so loss is computed only on label tokens.

Solution 3 Valid values are `all`, `base`, `lora`, `qlora`, `final`. Invalid values trigger a `ValueError` during config initialization.

Solution 4 QLoRA path sets `quantization_config=BitsAndBytesConfig(...)` with NF4 and optional fixed `device_map={"": 0}` in training mode. Non-quantized path uses `dtype=get_compute_dtype()` and standard model load.

Solution 5 Metrics dict: `run_name`, `samples_evaluated`, `accuracy_strict`, `macro_f1_known_only`, `label_parse_coverage`.
Predictions DataFrame: `text`, `true_label`, `pred_label`, `raw_generation`.

Solution 6 Transformers argument names differ by version (`eval_strategy` vs `evaluation_strategy`, and removed keys like `overwrite_output_dir`). Filtering prevents constructor errors across versions.

Solution 7 Each stage attempt logs to `artifacts/runs/<RUN_ID>/*.log`. If failure log matches OOM regex, script retries same stage at next smaller sequence length from ladder arrays. Non-OOM errors stop retries immediately.

Solution 8 Notebook uses `r=16, lora_alpha=32, lora_dropout=0.05, bias="none", task_type="CAUSAL_LM", target_modules="all-linear"`. Saved adapter configs resolve to concrete module names (`q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`) with matching hyperparameter values.

Learner Verification Checklist

Use this checklist to self-verify end-to-end understanding:

- Can you explain how `RUN_STAGE` controls execution and where it is validated?
- Can you describe the exact schema transformation from raw sample to collated training batch?
- Can you explain the difference between token-level eval (`Trainer.evaluate`) and generation-based eval (`evaluate_generation_classifier`)?
- Can you explain why cleanup requires nullifying notebook-scope references before `gc.collect()`?
- Can you compare LoRA vs QLoRA loading/training paths in concrete function calls?
- Can you explain how adapter files in `artifacts/*_adapter/` are sufficient for reload when combined with the same base model?
- Can you describe how `run_granite41_gpu_stages.sh` performs OOM retries and where logs are stored?
- Can you identify at least one repo-specific run caveat (for example hard-coded `PROJECT_ROOT`) and how to fix it?